



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

HLS4ML

High Level Synthesis for Machine Learning

Doctorado en Inteligencia Artificial

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Fast inference of deep neural networks in FPGAs

Implementación de ANNs en FPGA

Las redes neuronales en FPGA han sido aplicadas en sistemas con **restricciones de tiempo**, como:

- ▶ **Visión por computador**
- ▶ **Reconocimiento de voz**
- ▶ **Física de partículas**

Enfoques arquitectónicos

- ▶ **Motor de procesamiento único:** uso de *systolic arrays*¹ para acelerar operaciones Multiply-Accumulate (MACs). Procesamiento secuencial por capa.
- ▶ **Arquitectura de streaming:** cada capa tiene su propio motor; procesamiento en paralelo a costa de más recursos.
- ▶ **Procesador vectorial:** instrucciones personalizadas para operaciones de red (activaciones, pooling, etc.).

¹Es un grid de elementos de proceso que en sincronía, realizan MACs y se pasan los datos de vecino a vecino, maximizando el paralelismo y el reuso y minimizando el tráfico a memoria (En slides posteriores...)

Frameworks

1. **Overlay:** generan coprocesadores programables.

- ▶ MATLAB DLHDL, Vitis AI, VectorBlox.
- ▶ Requieren procesador host y librerías propietarias.
- ▶ Mayor flexibilidad, pero menos autonomía.

2. **Dedicated:** generan módulos IP autónomos.

- ▶ HLS4ML, FINN.
- ▶ No requieren host ni librerías externas.
- ▶ Más control, pero mayor responsabilidad de integración.

HLS4ML es una herramienta Dedicated que genera descripciones de hardware específicas desde modelos en alto nivel, facilitando la implementación directa sin dependencia de software propietario.



`hls4ml` is a Python package for machine learning inference in FPGAs. We create firmware implementations of ML algorithms using a *high-level synthesis* language (HLS).

`hls4ml` translate traditional open-source ML package models into HLS that can be configured for your use-case.

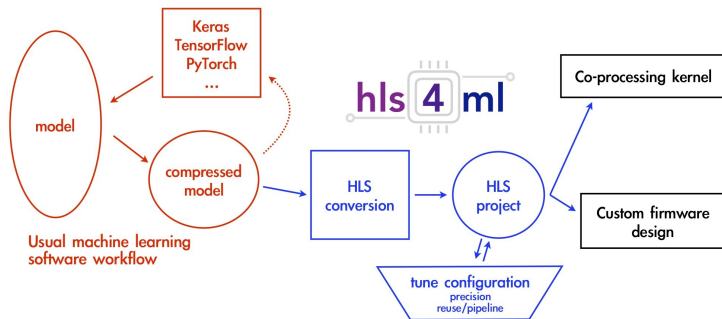
¿Qué es hls4ml?

hls4ml es una herramienta que traduce modelos de ML, entrenados con frameworks como Keras, TensorFlow o PyTorch, en implementaciones eficientes en hardware (FPGAs o ASICs), utilizando High-Level Synthesis.

1. Entrenamiento y compresión del modelo en el flujo habitual de ML.
2. Conversión del modelo a un proyecto HLS con hls4ml.
3. Ajuste de parámetros como *precisión*, *pipeline* y *reuse*.
4. Generación de firmware como **co-processing kernel** o diseño embebido personalizado.

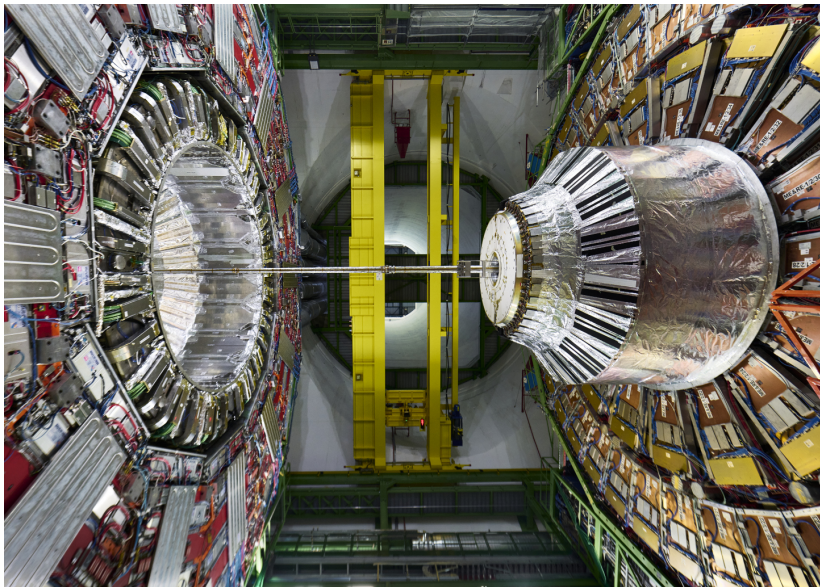
Ventajas!

Baja latencia, uso optimizado de recursos y despliegue eficiente en hardware. Ideal para aplicaciones en tiempo real y sistemas embebidos.



- Qkeras CPU
- Qkeras FPGA
- Post train quantization

hls4ml en el experimento CMS (CERN)



hls4ml en el experimento CMS (CERN)

En experimentos como CMS (*Compact Muon Solenoid*, uno de los experimentos del Colisionador de Hadrones) en el CERN, los detectores generan datos a una frecuencia de 40 MHz. Es necesario tomar decisiones en tiempo real (1 μ s) para descartar o retener eventos de colisión.

- ▶ Se entrena una ANN (ej. MLP) para clasificar eventos basados en características físicas (momentum, energía, ángulo, etc.).
- ▶ El modelo es convertido usando hls4ml a firmware para una FPGA (Xilinx Ultrascale+).
- ▶ El firmware se integra al **Level-1 (L1) trigger** del detector.

Resultados

- ▶ **Latencia total:** ~ 200 ns.
- ▶ **Recursos utilizados:** 5 % de LUTs, 3 % de BRAM.
- ▶ Precisión comparable con el modelo original en floating point.

Duarte et al., "Fast inference of deep neural networks in FPGAs for particle physics," (2018)

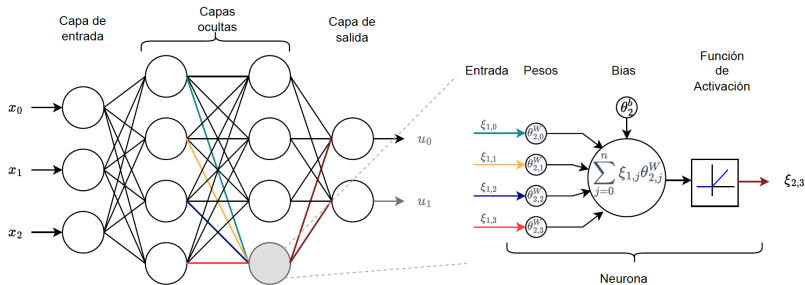


Figura 1: DNN de tres entradas, dos salidas, dos capas ocultas y cuatro neuronas por capa.

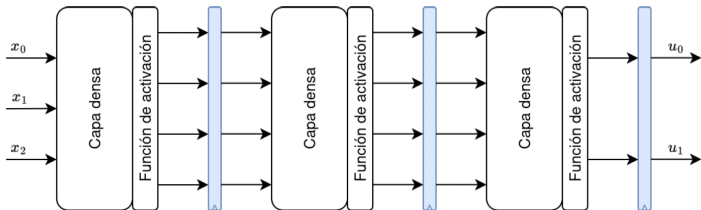


Figura 2: Vista de alto nivel de la implementación con HLS4ML de una ANN

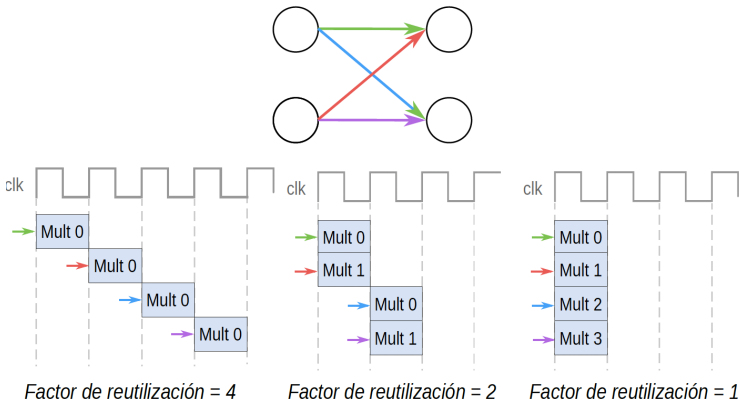


Figura 3: Factor de reutilización para una capa densa en HLS4ML.

Parámetros clave de configuración: ReuseFactor y Cuantización

ReuseFactor: paralelismo y uso de recursos

Define cuántas veces se reutiliza un multiplicador dentro de una capa:

- ▶ **RF = 4:** 1 multiplicador, 4 ciclos $\Rightarrow$ mínimo recurso, mayor latencia.
- ▶ **RF = 2:** 2 multiplicadores, 2 ciclos.
- ▶ **RF = 1:** 4 multiplicadores, 1 ciclo $\Rightarrow$ máximo rendimiento.

Cada capa tiene sus propios multiplicadores; no se comparten entre capas.

Cuantización: precisión vs. recursos

HLS4ML usa representaciones en **punto fijo** definidas por:

- ▶ W = total de bits (ej. 8)
- ▶ Q = bits para parte entera + signo (ej. 4)

Ejemplo: `ap_fixed<8,4>` representa valores entre -8 y $7,9375$.

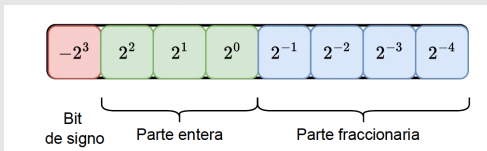


Figura 4: Representación en punto fijo de 8 bits, con 3 bits para parte entera.

Compatible con QKeras para preservar la cuantización desde el entrenamiento.

Verificación

- ▶ Se usa una **golden reference** para validar el código C/C++ generado.
- ▶ La salida del modelo HLS se compara con las salidas esperadas del modelo original.
- ▶ Se realiza **cosimulación** para verificar la equivalencia funcional entre el modelo HLS (C++) y la implementación HDL (VHDL/Verilog).

Salida del flujo de diseño

- ▶ **IP-package**: módulo integrable con otros bloques del sistema.
- ▶ **Bitstream**: imagen para configuración directa de la FPGA.

Ventaja

HLS4ML permite iterar en configuraciones de hardware desde alto nivel, reduciendo drásticamente el tiempo de prototipado comparado con diseño manual en RTL.



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

HLS4ML

High Level Synthesis for Machine Learning

Doctorado en Inteligencia Artificial

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile